WEEK 6

State & User Input

Daily study material with detailed examples

Android Development with Kotlin | Prepared for **Anjana**

Until now your screens were **static**: they looked the same no matter what the user did. This week they come alive. You'll learn **state**, the values that change and make the UI update, plus how to read text input, handle rotation, and structure state properly with **state hoisting**. You'll finish by building a **Tip Calculator** and a **Dice Roller**.

Day	Topic	You will be able to...
Day 1	State & recomposition	Make the UI react to changes with remember and mutableStateOf
Day 2	rememberSaveable & the lifecycle	Keep state when the phone rotates; understand Activity lifecycle
Day 3	Text input	Read and validate what the user types
Day 4	State hoisting	Write reusable stateless composables, the professional way
Day 5	Interactive components	Use checkboxes, switches, sliders, dropdowns, dialogs and animations
Day 6	Weekly projects	Build a Tip Calculator and a Dice Roller

Tip: Compose examples are **compile-checked** (imports left out: use Alt + Enter). Pure Kotlin logic (like the tip calculation) was **run**, and the output shown is real. Practise in a new project, **Week6Practice**.

WEEK 6 • DAY 1

State & Recomposition

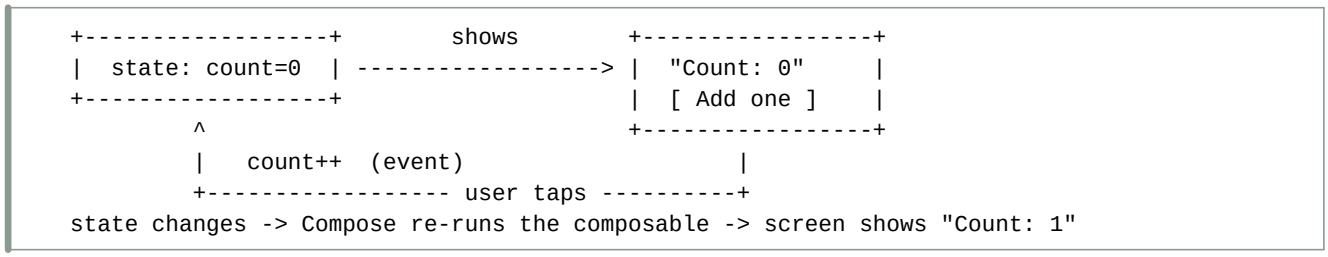
Today's goal: Understand state, make composables react to changes, and know what recomposition is.

1. What is state?

State is any value that can **change over time** and that the UI **shows**: a counter, the text in a search box, whether a switch is on, the list of items in a cart.

In Compose, the UI is a **function of state**: `UI = f(state)`. When the state changes, Compose calls your composable function again with the new value and updates the screen. This re-run is called **recomposition**.

Sketch: The state loop



2. Your first state

A working counter

```
@Composable
fun Counter() {
    var count by remember { mutableStateOf(0) }

    Column(
        modifier = Modifier.padding(24.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text("Count: $count", style = MaterialTheme.typography.headlineMedium)
        Spacer(Modifier.height(16.dp))
        Button(onClick = { count++ }) {
            Text("Add one")
        }
    }
}
```

Part	What it does
mutableStateOf(0)	Creates an **observable** value starting at 0. Compose watches it, and any composable that reads it recomposes when it changes
remember { ... }	Keeps the value **across recompositions**. Without it, every re-run would create a new state starting at 0
var count by ...	Kotlin **property delegation**: lets you write **count** instead of **count.value**
count++	Changes the state, so Compose schedules a recomposition

Tip: If **by** shows a red error, press Alt + Enter and import **androidx.compose.runtime.getValue** and **setValue**.

3. The two classic mistakes

BUG 1: a normal variable. Compose doesn't know it changed

```
@Composable
fun BrokenCounter1() {
    var count = 0                          // not state
    Button(onClick = { count++ }) {        // count changes, but nothing recomposes
        Text("Count: $count")              // always shows 0
    }
}
```

BUG 2: state without remember. It resets on every recomposition

```
@Composable
fun BrokenCounter2() {
    var count by mutableStateOf(0)         // Android Studio warns: "Creating a state object during composition without using remember"
    Button(onClick = { count++ }) {
        Text("Count: $count")
    }
}
```

4. Several pieces of state

Like button with a counter

```
@Composable
fun LikeButton() {
    var liked by remember { mutableStateOf(false) }
    var likes by remember { mutableStateOf(41) }

    Row(verticalAlignment = Alignment.CenterVertically, modifier = Modifier.padding(16.dp)) {
        IconButton(onClick = {
            liked = !liked
            if (liked) likes++ else likes--
        }) {
            Icon(
                imageVector = if (liked) Icons.Filled.Favorite else Icons.Outlined.FavoriteBorder,
                contentDescription = if (liked) "Unlike" else "Like",
                tint = if (liked) MaterialTheme.colorScheme.error else MaterialTheme.colorScheme.onSurface
            )
        }
        Text("$likes likes")
    }
}
```

5. Derived values: calculate, don't store

If a value can be **calculated** from other state, don't make it a separate state. Calculate it in the composable. Then it can never get out of sync.

Derived values

```
@Composable
fun CartSummary() {
    var quantity by remember { mutableStateOf(1) }
    val pricePerItem = 120
    val total = quantity * pricePerItem            // derived: recalculated on every recomposition
    val freeDelivery = total >= 500                 // derived too

    Column(modifier = Modifier.padding(16.dp)) {
        Row(verticalAlignment = Alignment.CenterVertically) {
            OutlinedButton(onClick = { if (quantity > 1) quantity-- }) { Text("-") }
            Text("  $quantity  ", style = MaterialTheme.typography.titleLarge)
            OutlinedButton(onClick = { quantity++ }) { Text("+") }
        }
        Text("Total: Rs $total")
        if (freeDelivery) {
            Text("Free delivery!", color = MaterialTheme.colorScheme.primary)
        } else {
            Text("Add Rs ${500 - total} more for free delivery")
        }
    }
}
```

6. What exactly recomposes?

- Only composables that **read** the changed state are re-run. Compose skips the others. This is why Compose is fast.
- Recomposition can happen **often** and in **any order**, so a composable must not do slow work (network calls, heavy loops) or change things outside itself. Just describe the UI.
- Event handlers like **onClick** are **not** part of composition: they run only when the event happens. That's the right place to change state.

Practice Exercises

1. Build a **StepCounter** with +1, +10 and Reset buttons.

2. Build a **LightSwitch**: a Button that toggles a Boolean state, and a Box that is yellow (tertiaryContainer) when on and grey (surfaceVariant) when off.

3. Build a **WaterTracker**: tap a button to add a 250 ml glass; show total ml and a derived message "Goal reached!" at 2000 ml.

4. Explain in your own words why BrokenCounter1 and BrokenCounter2 don't work.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What is recomposition?

Q2. What does remember do?

Q3. Why does a normal var not update the UI?

Q4. Should the total price be a separate state?

Q5. Where should you change state: in the composable body or in onClick?

WEEK 6 • DAY 2

rememberSaveable & the Activity Lifecycle

Today's goal: Keep state through rotation, understand the Activity lifecycle, and store lists in state correctly.

1. The rotation problem

Run the Day 1 counter, tap it a few times, then **rotate the phone**. The count goes back to 0! On rotation (and other **configuration changes** like changing language or dark mode), Android **destroys and recreates** the Activity, and **remember** forgets everything.

rememberSaveable survives rotation

```
@Composable
fun SaveableCounter() {
    var count by rememberSaveable { mutableStateOf(0) }    // only this word changed

    Button(onClick = { count++ }, modifier = Modifier.padding(16.dp)) {
        Text("Count: $count")
    }
}
```

	remember	rememberSaveable
Survives recomposition	Yes	Yes
Survives rotation / configuration change	No	**Yes**
Survives the app being killed in the background	No	Yes (Android restores it)
Can store	Anything	Simple types (Int, String, Boolean, lists of them...) or Parcelable objects
Use for	Temporary UI things (animations, objects)	What the user typed or chose

Tip: Rule of thumb: if the user would be **annoyed to lose it** (text they typed, a tab they selected), use **rememberSaveable**. In Week 8 you'll learn the ViewModel, which is the main way apps keep screen data through rotation.

2. The Activity lifecycle

Sketch: Lifecycle callbacks

```
onCreate()   -> the Activity is created: setContent { } runs here
onStart()    -> becoming visible
onResume()   -> in front, the user can interact          <--+
     (user presses Home or another app opens)               |
onPause()    -> losing focus                                |  user returns:
onStop()     -> no longer visible                           |  onRestart -> onStart -> onResume
onDestroy()  -> being destroyed (closed, or ROTATED)   ------+
```

What happens	Callbacks called
App opens	onCreate → onStart → onResume
User presses Home	onPause → onStop
User comes back	onRestart → onStart → onResume

What happens	Callbacks called
Phone rotates	onPause → onStop → onDestroy → **onCreate** → onStart → onResume
User presses Back on the last screen	onPause → onStop → onDestroy

See it yourself in Logcat (Android only)

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d("Lifecycle", "onCreate")
        setContent { Week6PracticeTheme { SaveableCounter() } }
    }
    override fun onStart() { super.onStart(); Log.d("Lifecycle", "onStart") }
    override fun onResume() { super.onResume(); Log.d("Lifecycle", "onResume") }
    override fun onPause() { super.onPause(); Log.d("Lifecycle", "onPause") }
    override fun onStop() { super.onStop(); Log.d("Lifecycle", "onStop") }
    override fun onDestroy() { super.onDestroy(); Log.d("Lifecycle", "onDestroy") }
}
```

Filter Logcat by **tag:Lifecycle**, then open the app, press Home, come back and rotate. Watch the order of the messages.

3. Lists in state

BUG: changing a normal list doesn't recompose

```
val items = remember { mutableListOf("Milk") }   // a normal MutableList: not observable
Button(onClick = { items.add("Bread") }) { }     // the list changes, the UI doesn't
```

Two correct ways to keep a list in state

```
@Composable
fun ShoppingListA() {
    // Way 1: an observable list. add/remove trigger recomposition
    val items = remember { mutableStateListOf("Milk", "Eggs") }

    Column(modifier = Modifier.padding(16.dp)) {
        items.forEach { Text("- $it") }
        Button(onClick = { items.add("Item ${items.size + 1}") }) { Text("Add item") }
    }
}

@Composable
fun ShoppingListB() {
    // Way 2: a read-only list in state. Replace it with a NEW list
    var items by rememberSaveable { mutableStateOf(listOf("Milk", "Eggs")) }

    Column(modifier = Modifier.padding(16.dp)) {
        items.forEach { Text("- $it") }
        Button(onClick = { items = items + "Item ${items.size + 1}" }) { Text("Add item") }
    }
}
```

Way 2 (a **new immutable list** each time) is what ViewModels and UiState use, so get used to it: `items + newItem`, `items - oldItem`, `items.filter { }`.

4. Keeping custom objects

Saving a data class with rememberSaveable

```
data class Filter(val query: String = "", val onlyFavourites: Boolean = false)

@Composable
fun FilterBar() {
    var filter by remember { mutableStateOf(Filter()) }        // remember is fine for objects

    Row(verticalAlignment = Alignment.CenterVertically, modifier = Modifier.padding(16.dp)) {
        Checkbox(
            checked = filter.onlyFavourites,
            onCheckedChange = { filter = filter.copy(onlyFavourites = it) }   // copy, don't mutate
        )
        Text("Only favourites")
    }
}
```

To use **rememberSaveable** with your own class, mark it **@Parcelize** and implement Parcelable (Android only), or simply store the simple values separately. In real apps this kind of screen state usually lives in a ViewModel (Week 8).

Practice Exercises

1. Change your Day 1 projects to use rememberSaveable. Rotate the phone to check that they keep their values.

2. Add lifecycle logs to MainActivity and write down the order of callbacks for: opening the app, pressing Home, returning and rotating.

3. Build a **TodoQuickList** with mutableStateListOf: a button adds "Task N", and each task has a Remove button.

4. Rewrite TodoQuickList using an immutable list in state (items = items - task).

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. Why does remember lose its value when you rotate the phone?

Q2. What should you use for text the user typed?

Q3. Which lifecycle callbacks run when the phone rotates?

Q4. Why doesn't mutableListOf inside remember update the UI?

Q5. Name two correct ways to keep a list in state.

WEEK 6 • DAY 3

Text Input

Today's goal: Read, configure and validate user text input with TextField and OutlinedTextField.

1. A text field is controlled by state

The basic pattern

```
@Composable
fun NameInput() {
    var name by rememberSaveable { mutableStateOf("") }

    Column(modifier = Modifier.padding(16.dp)) {
        OutlinedTextField(
            value = name,                       // 1. show the current state
            onValueChange = { name = it },      // 2. update the state when the user types
            label = { Text("Your name") },
            singleLine = true,
            modifier = Modifier.fillMaxWidth()
        )
        Spacer(Modifier.height(8.dp))
        Text(if (name.isBlank()) "Hello, stranger!" else "Hello, $name!")
    }
}
```

Careful: If you forget **name = it** in onValueChange, the field **won't change** when you type. The TextField only shows what **value** says. It doesn't store text itself.

2. Decorating text fields

Labels, placeholders, icons and helper text

```
@Composable
fun DecoratedField() {
    var city by rememberSaveable { mutableStateOf("") }

    OutlinedTextField(
        value = city,
        onValueChange = { city = it },
        label = { Text("City") },
        placeholder = { Text("e.g. Kochi") },
        leadingIcon = { Icon(Icons.Default.LocationOn, contentDescription = null) },
        trailingIcon = {
            if (city.isNotEmpty()) {
                IconButton(onClick = { city = "" }) {
                    Icon(Icons.Default.Clear, contentDescription = "Clear")
                }
            }
        },
        supportingText = { Text("${city.length} / 30") },
        singleLine = true,
        modifier = Modifier.fillMaxWidth().padding(16.dp)
    )
}
```

3. Keyboard types and actions

Choosing the right keyboard

```
@Composable
fun KeyboardExamples() {
    var phone by rememberSaveable { mutableStateOf("") }
    var amount by rememberSaveable { mutableStateOf("") }
    val focusManager = LocalFocusManager.current

    Column(modifier = Modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(8.dp)) {
        OutlinedTextField(
            value = phone,
            onValueChange = { phone = it },
            label = { Text("Phone") },
            singleLine = true,
            keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Phone, imeAction = ImeAction.Next),
            modifier = Modifier.fillMaxWidth()
        )
        OutlinedTextField(
            value = amount,
            onValueChange = { amount = it },
            label = { Text("Amount (Rs)") },
            singleLine = true,
            keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number, imeAction = ImeAction.Done),
            keyboardActions = KeyboardActions(onDone = { focusManager.clearFocus() }),  // hide keyboard
            modifier = Modifier.fillMaxWidth()
        )
    }
}
```

KeyboardType	Shows
Text (default)	Normal keyboard
Number / Decimal	Number pad
Phone	Phone dial pad
Email	Keyboard with @ and .com
Password	Hides suggestions; combine with PasswordVisualTransformation()

imeAction sets the keyboard's action button (Next, Done, Search, Send). **keyboardActions** says what happens when it's pressed. With Next, focus moves to the next field automatically.

4. Numbers from text

Safely converting input to numbers

```
@Composable
fun AgeChecker() {
    var ageText by rememberSaveable { mutableStateOf("") }
    val age = ageText.toIntOrNull()                    // null if empty or not a number

    Column(modifier = Modifier.padding(16.dp)) {
        OutlinedTextField(
            value = ageText,
            onValueChange = { newText ->
                if (newText.all { it.isDigit() } && newText.length <= 3) ageText = newText
            },
            label = { Text("Age") },
            keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
            singleLine = true
        )
        Text(
            when {
                age == null -> "Enter your age"
                age >= 18 -> "You can vote!"
                else -> "You can vote in ${18 - age} years"
            }
        )
    }
}
```

Notice the filter inside onValueChange: only digits are accepted, and at most 3 of them. **toIntOrNull()** (Week 1) means an empty field never crashes the app.

5. Validation and error messages

An email field with validation

```
@Composable
fun EmailField() {
    var email by rememberSaveable { mutableStateOf("") }
    var touched by rememberSaveable { mutableStateOf(false) }
    val isValid = email.contains("@") && email.substringAfter("@").contains(".")
    val showError = touched && !isValid

    OutlinedTextField(
        value = email,
        onValueChange = {
            email = it.trim()
            touched = true
        },
        label = { Text("Email") },
        isError = showError,
        supportingText = { if (showError) Text("Enter a valid email address") },
        singleLine = true,
        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Email),
        modifier = Modifier.fillMaxWidth().padding(16.dp)
    )
}
```

Tip: Good UX: don't show an error before the user has typed anything. That's what **touched** is for. Many apps validate only when the user taps Submit.

6. Password field

Password with show/hide

```
@Composable
fun PasswordField() {
    var password by rememberSaveable { mutableStateOf("") }
    var visible by rememberSaveable { mutableStateOf(false) }

    OutlinedTextField(
        value = password,
        onValueChange = { password = it },
        label = { Text("Password") },
        singleLine = true,
        visualTransformation = if (visible) VisualTransformation.None else PasswordVisualTransformation(),
        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Password),
        trailingIcon = {
            TextButton(onClick = { visible = !visible }) { Text(if (visible) "Hide" else "Show") }
        },
        modifier = Modifier.fillMaxWidth().padding(16.dp)
    )
}
```

Practice Exercises

1. Build a **BmiInput** screen: weight (kg) and height (cm) fields with number keyboards, and the BMI (rounded to 1 decimal) shown below when both are valid.

2. Build a **FeedbackField**: a multi-line field (minLines = 3, maxLines = 5) with a 200-character limit and a counter.

3. Add a "Confirm password" field to PasswordField with an error when the two don't match.

4. Build a phone number field that only accepts digits, at most 10, and shows an error until there are exactly 10.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What happens if onValueChange doesn't update the state?

Q2. Which option shows a number pad?

Q3. How do you hide the keyboard when the user presses Done?

Q4. Why use toIntOrNull() for number input?

Q5. Which parameters show an error state?

WEEK 6 • DAY 4

State Hoisting

Today's goal: Make composables stateless and reusable by moving state up, and keep one source of truth.

1. Stateful vs stateless composables

Stateful	Stateless
Owns its state (remember inside)	Receives the value as a parameter
Easy to use, hard to control from outside	Easy to reuse, preview and test
Example: Counter() with remember inside	Example: Counter(count, onIncrement)

State hoisting means moving state **up** to a parent, and replacing it with two parameters: the **value** and an **event callback**. Rule: **state flows down, events flow up**.

2. Hoisting a counter

Before and after

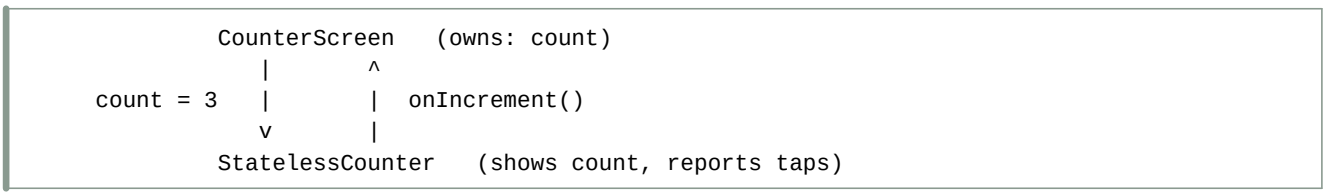
```
// BEFORE: stateful. The parent can't read or reset the count
@Composable
fun StatefulCounter() {
    var count by rememberSaveable { mutableStateOf(0) }
    Button(onClick = { count++ }) { Text("Count: $count") }
}

// AFTER: stateless. It shows what it's given and reports clicks
@Composable
fun StatelessCounter(count: Int, onIncrement: () -> Unit, modifier: Modifier = Modifier) {
    Button(onClick = onIncrement, modifier = modifier) { Text("Count: $count") }
}

// The parent owns the state, so it can share it and reset it
@Composable
fun CounterScreen() {
    var count by rememberSaveable { mutableStateOf(0) }

    Column(modifier = Modifier.padding(16.dp)) {
        StatelessCounter(count = count, onIncrement = { count++ })
        Text(if (count >= 10) "Wow, that's a lot of taps!" else "Keep tapping")
        TextButton(onClick = { count = 0 }) { Text("Reset") }
    }
}
```

Sketch: State down, events up



3. Where should state live?

Hoist state to the **lowest common parent** of all the composables that **read** or **change** it. Not higher, not lower.

Two composables sharing one state

```
@Composable
fun TemperatureConverter() {
    var celsiusText by rememberSaveable { mutableStateOf("") }
    val fahrenheit = celsiusText.toDoubleOrNull()?.let { it * 9 / 5 + 32 }

    Column(modifier = Modifier.padding(16.dp)) {
        CelsiusInput(value = celsiusText, onValueChange = { celsiusText = it })
        Spacer(Modifier.height(8.dp))
        FahrenheitResult(fahrenheit = fahrenheit)
    }
}

@Composable
fun CelsiusInput(value: String, onValueChange: (String) -> Unit) {
    OutlinedTextField(
        value = value,
        onValueChange = onValueChange,
        label = { Text("Celsius") },
        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
        singleLine = true
    )
}

@Composable
fun FahrenheitResult(fahrenheit: Double?) {
    Text(
        text = if (fahrenheit == null) "Enter a temperature" else "= ${"%.1f".format(fahrenheit)} F",
        style = MaterialTheme.typography.titleLarge
    )
}
```

CelsiusInput **changes** the value and FahrenheitResult **reads** it, so the state lives in their common parent, TemperatureConverter. Both children are stateless, so you can preview them with any value.

4. A reusable, hoisted component

QuantitySelector

```
@Composable
fun QuantitySelector(
    quantity: Int,
    onQuantityChange: (Int) -> Unit,
    modifier: Modifier = Modifier,
    min: Int = 1,
    max: Int = 10
) {
    Row(modifier = modifier, verticalAlignment = Alignment.CenterVertically) {
        IconButton(onClick = { onQuantityChange(quantity - 1) }, enabled = quantity > min) {
            Icon(Icons.Default.KeyboardArrowDown, contentDescription = "Decrease")
        }
        Text("$quantity", style = MaterialTheme.typography.titleMedium)
        IconButton(onClick = { onQuantityChange(quantity + 1) }, enabled = quantity < max) {
            Icon(Icons.Default.KeyboardArrowUp, contentDescription = "Increase")
        }
    }
}

@Preview(showBackground = true)
@Composable
fun QuantitySelectorPreview() {
    QuantitySelector(quantity = 3, onQuantityChange = { })
}
```

Tip: Look at Material components: **Checkbox(checked, onCheckedChange)**, **TextField(value, onValueChange)**, **Switch(checked, onCheckedChange)**. They're all hoisted! Your own components should follow the same pattern.

5. Single source of truth

BAD: two copies of the same state get out of sync

```
var cartCount by remember { mutableStateOf(0) }       // in the screen
var badgeCount by remember { mutableStateOf(0) }      // in the top bar: a second copy!
// Adding an item updates cartCount but someone forgets badgeCount -> wrong badge
```

Keep each piece of state in **one place** and pass it down to everyone who needs it. In Week 8, that one place for a whole screen becomes the **ViewModel**.

Practice Exercises

1. Hoist the state out of the LikeButton from Day 1: **LikeButton(liked, likes, onToggle)**, with the state in a parent.
2. Build a stateless **RatingBar(rating: Int, onRatingChange: (Int) -> Unit)** with 5 star IconButtons, and a parent that shows "You rated N/5".
3. Build a **CurrencyConverter**: a stateless input for rupees and a stateless result showing dollars (1 USD = 84 INR), with the state in the parent.
4. Use QuantitySelector twice on one screen (adults and children) and show the total ticket price (adult Rs 200, child Rs 100).

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What is state hoisting?

Q2. Where should hoisted state live?

Q3. Name two benefits of stateless composables.

Q4. What does "single source of truth" mean?

Q5. Which two parameters replace hoisted state?

WEEK 6 • DAY 5

Interactive Components

Today's goal: Use checkboxes, switches, radio buttons, sliders, dropdown menus, dialogs and simple animations.

1. Checkbox, Switch and RadioButton

Preferences form

```
@Composable
fun PreferencesForm() {
    var newsletter by rememberSaveable { mutableStateOf(false) }
    var darkMode by rememberSaveable { mutableStateOf(false) }
    var language by rememberSaveable { mutableStateOf("English") }

    Column(modifier = Modifier.padding(16.dp)) {
        Row(
            verticalAlignment = Alignment.CenterVertically,
            modifier = Modifier.fillMaxWidth().clickable { newsletter = !newsletter }
        ) {
            Checkbox(checked = newsletter, onCheckedChange = { newsletter = it })
            Text("Send me the newsletter")
        }
        Row(verticalAlignment = Alignment.CenterVertically) {
            Text("Dark mode", modifier = Modifier.weight(1f))
            Switch(checked = darkMode, onCheckedChange = { darkMode = it })
        }
        Text("Language", style = MaterialTheme.typography.titleSmall, modifier = Modifier.padding(top = 16.dp))
        listOf("English", "Malayalam", "Hindi").forEach { option ->
            Row(
                verticalAlignment = Alignment.CenterVertically,
                modifier = Modifier.fillMaxWidth().clickable { language = option }
            ) {
                RadioButton(selected = language == option, onClick = { language = option })
                Text(option)
            }
        }
    }
}
```

Tip: Make the **whole row** clickable, not just the small checkbox or radio button. It's much easier to tap.

2. Slider

Choosing a value with a slider

```
@Composable
fun VolumeSlider() {
    var volume by rememberSaveable { mutableStateOf(50f) }

    Column(modifier = Modifier.padding(16.dp)) {
        Text("Volume: ${volume.roundToInt()}%")
        Slider(
            value = volume,
            onValueChange = { volume = it },
            valueRange = 0f..100f,
            steps = 9                          // 9 steps between = values in jumps of 10
        )
    }
}
```

3. Dropdown menu

Exposed dropdown (a select box)

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun CityDropdown(selected: String, onSelect: (String) -> Unit) {
    val cities = listOf("Kochi", "Thiruvananthapuram", "Kozhikode", "Thrissur")
    var expanded by remember { mutableStateOf(false) }

    ExposedDropdownMenuBox(expanded = expanded, onExpandedChange = { expanded = it }) {
        OutlinedTextField(
            value = selected,
            onValueChange = { },
            readOnly = true,
            label = { Text("City") },
            trailingIcon = { ExposedDropdownMenuDefaults.TrailingIcon(expanded = expanded) },
            modifier = Modifier.menuAnchor(MenuAnchorType.PrimaryNotEditable).fillMaxWidth()
        )
        ExposedDropdownMenu(expanded = expanded, onDismissRequest = { expanded = false }) {
            cities.forEach { city ->
                DropdownMenuItem(
                    text = { Text(city) },
                    onClick = {
                        onSelect(city)
                        expanded = false
                    }
                )
            }
        }
    }
}
```

expanded is local UI state (is the menu open?), so it stays inside with remember. The **selected city** is hoisted because the parent needs it.

4. Dialogs controlled by state

Showing a dialog with a Boolean

```
@Composable
fun LogoutButton(onLogout: () -> Unit) {
    var showDialog by remember { mutableStateOf(false) }

    Button(onClick = { showDialog = true }) { Text("Log out") }

    if (showDialog) {
        AlertDialog(
            onDismissRequest = { showDialog = false },
            title = { Text("Log out?") },
            text = { Text("You'll need to log in again to see your data.") },
            confirmButton = {
                TextButton(onClick = {
                    showDialog = false
                    onLogout()
                }) { Text("Log out") }
            },
            dismissButton = {
                TextButton(onClick = { showDialog = false }) { Text("Cancel") }
            }
        )
    }
}
```

In Compose, a dialog is shown simply by **being in the composition**. When showDialog becomes false, the **if** stops calling AlertDialog, and the dialog disappears.

5. Simple animations

AnimatedVisibility and animated values

```
@Composable
fun ExpandableFaq(question: String, answer: String) {
    var expanded by rememberSaveable { mutableStateOf(false) }
    val arrowRotation by animateFloatAsState(targetValue = if (expanded) 180f else 0f, label = "arrow")

    Card(modifier = Modifier.fillMaxWidth().padding(16.dp).clickable { expanded = !expanded }) {
        Column(modifier = Modifier.padding(16.dp)) {
            Row(verticalAlignment = Alignment.CenterVertically) {
                Text(question, style = MaterialTheme.typography.titleMedium, modifier = Modifier.weight(1f))
                Icon(
                    Icons.Default.KeyboardArrowDown,
                    contentDescription = if (expanded) "Collapse" else "Expand",
                    modifier = Modifier.rotate(arrowRotation)
                )
            }
            AnimatedVisibility(visible = expanded) {
                Text(answer, modifier = Modifier.padding(top = 8.dp))
            }
        }
    }
}
```

API	Animates
AnimatedVisibility(visible)	Showing and hiding content (fade + expand by default)
animateFloatAsState / animateDpAsState / animateColorAsState	A single value smoothly moving to a new target
Modifier.animateContentSize()	A container's size when its content changes

API	Animates
Crossfade(targetState)	Fading between two different contents

Practice Exercises

1. Build a **PizzaOrder** form: size (radio buttons), extra cheese (checkbox), spicy level (slider 0-3 with steps = 2) and a summary Text.

2. Use CityDropdown in a parent that shows "Delivering to: CITY".

3. Build a "Delete account" button whose dialog asks the user to type DELETE in a text field before the confirm button is enabled.

4. Make a list of 3 ExpandableFaq cards about your app.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. Why make the whole row clickable for a checkbox?

Q2. What does steps = 9 mean on a 0..100 slider?

Q3. How do you show and hide an AlertDialog?

Q4. Which state in CityDropdown is local and which is hoisted?

Q5. Which API animates content appearing and disappearing?

WEEK 6 • DAY 6

Weekly Projects: Tip Calculator & Dice Roller

Today's goal: Combine state, text input, hoisting and interactive components in two complete apps.

PROJECT 1: Tip Calculator

Sketch: The design

```
+-----------------------------------+
| Tip Calculator                    |
| +-------------------------------+ |
| | (cart) Bill amount (Rs)  1250 | |
| +-------------------------------+ |
| Tip: 15%                          |
| o-----------O-----------------o   |   <- Slider 0..30 in steps of 5
| Round up tip                [on]  |   <- Switch
| Split between          (v)  3  (^)  |   <- QuantitySelector
| +-------------------------------+ |
| | Tip: Rs 188.00                | |
| | Total: Rs 1438.00             | |
| | Each person pays Rs 479.33    | |
| +-------------------------------+ |
+-----------------------------------+
```

Step 1: the logic, as plain Kotlin

Professional developers keep **calculations out of composables**, in plain functions. They're easy to test without a phone:

TipLogic.kt (tested with real output)

```
fun calculateTip(amount: Double, tipPercent: Double, roundUp: Boolean): Double {
    var tip = amount * tipPercent / 100
    if (roundUp) tip = kotlin.math.ceil(tip)
    return tip
}

fun formatRupees(value: Double): String = "Rs " + String.format("%.2f", value)

fun main() {
    println(formatRupees(calculateTip(1250.0, 15.0, roundUp = false)))
    println(formatRupees(calculateTip(1250.0, 15.0, roundUp = true)))
    println(formatRupees(calculateTip(999.0, 10.0, roundUp = true)))
    println(formatRupees(calculateTip(0.0, 20.0, roundUp = false)))
}
```

Output

```
Rs 187.50
Rs 188.00
Rs 100.00
Rs 0.00
```

Step 2: the screen

TipCalculatorScreen.kt

```
@Composable
fun TipCalculatorScreen(modifier: Modifier = Modifier) {
    var amountInput by rememberSaveable { mutableStateOf("") }
    var tipPercent by rememberSaveable { mutableStateOf(15f) }
    var roundUp by rememberSaveable { mutableStateOf(false) }
    var people by rememberSaveable { mutableStateOf(1) }

    val amount = amountInput.toDoubleOrNull() ?: 0.0
    val tip = calculateTip(amount, tipPercent.toDouble(), roundUp)
    val total = amount + tip
    val perPerson = total / people

    Column(
        modifier = modifier
            .fillMaxSize()
            .verticalScroll(rememberScrollState())
            .padding(24.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ) {
        Text("Tip Calculator", style = MaterialTheme.typography.headlineMedium)

        OutlinedTextField(
            value = amountInput,
            onValueChange = { amountInput = it },
            label = { Text("Bill amount (Rs)") },
            leadingIcon = { Icon(Icons.Default.ShoppingCart, contentDescription = null) },
            singleLine = true,
            keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
            modifier = Modifier.fillMaxWidth()
        )

        Text("Tip: ${tipPercent.roundToInt()}%")
        Slider(value = tipPercent, onValueChange = { tipPercent = it }, valueRange = 0f..30f, steps = 5)

        Row(verticalAlignment = Alignment.CenterVertically) {
            Text("Round up tip", modifier = Modifier.weight(1f))
            Switch(checked = roundUp, onCheckedChange = { roundUp = it })
        }

        Row(verticalAlignment = Alignment.CenterVertically) {
            Text("Split between", modifier = Modifier.weight(1f))
            QuantitySelector(quantity = people, onQuantityChange = { people = it }, max = 20)
        }

        ResultCard(tip = tip, total = total, perPerson = perPerson, people = people)
    }
}

@Composable
fun ResultCard(tip: Double, total: Double, perPerson: Double, people: Int) {
    ElevatedCard(modifier = Modifier.fillMaxWidth()) {
        Column(modifier = Modifier.padding(20.dp), verticalArrangement = Arrangement.spacedBy(4.dp)) {
            Text("Tip: ${formatRupees(tip)}", style = MaterialTheme.typography.bodyLarge)
            Text("Total: ${formatRupees(total)}", style = MaterialTheme.typography.titleLarge)
            if (people > 1) {
                Text("Each person pays ${formatRupees(perPerson)}",
                    style = MaterialTheme.typography.titleMedium,
                    color = MaterialTheme.colorScheme.primary)
            }
        }
    }
}
```

```
@Composable
fun QuantitySelector(quantity: Int, onQuantityChange: (Int) -> Unit, min: Int = 1, max: Int = 10) {
    Row(verticalAlignment = Alignment.CenterVertically) {
        IconButton(onClick = { onQuantityChange(quantity - 1) }, enabled = quantity > min) {
            Icon(Icons.Default.KeyboardArrowDown, contentDescription = "Fewer people")
        }
        Text("$quantity", style = MaterialTheme.typography.titleMedium)
        IconButton(onClick = { onQuantityChange(quantity + 1) }, enabled = quantity < max) {
            Icon(Icons.Default.KeyboardArrowUp, contentDescription = "More people")
        }
    }
}
```

Concept	Where you see it
State with rememberSaveable	amountInput, tipPercent, roundUp, people survive rotation
Derived values	amount, tip, total and perPerson are calculated, not stored
Safe number input	amountInput.toDoubleOrNull() ?: 0.0
State hoisting	QuantitySelector and ResultCard are stateless
Logic outside the UI	calculateTip and formatRupees are plain functions
Scrolling for small screens	verticalScroll(rememberScrollState())

PROJECT 2: Dice Roller

DiceRollerScreen.kt

```
@Composable
fun DiceRollerScreen(modifier: Modifier = Modifier) {
    var dice by rememberSaveable { mutableStateOf(listOf(1, 1)) }
    var rolls by rememberSaveable { mutableStateOf(0) }
    val total = dice.sum()

    Column(
        modifier = modifier.fillMaxSize().padding(24.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Row(horizontalArrangement = Arrangement.spacedBy(16.dp)) {
            dice.forEach { value -> DieFace(value) }
        }
        Spacer(Modifier.height(24.dp))
        Text("Total: $total", style = MaterialTheme.typography.headlineMedium)
        if (dice.distinct().size == 1 && rolls > 0) {
            Text("Doubles!", color = MaterialTheme.colorScheme.primary,
                style = MaterialTheme.typography.titleLarge)
        }
        Spacer(Modifier.height(24.dp))
        Button(onClick = {
            dice = List(dice.size) { Random.nextInt(1, 7) }
            rolls++
        }) {
            Text("Roll")
        }
        TextButton(onClick = { dice = if (dice.size == 2) listOf(1) else listOf(1, 1) }) {
            Text(if (dice.size == 2) "Use one die" else "Use two dice")
        }
        Text("Rolls: $rolls", style = MaterialTheme.typography.bodySmall)
    }
}

@Composable
```

```
fun DieFace(value: Int) {
    Box(
        modifier = Modifier
            .size(96.dp)
            .clip(RoundedCornerShape(16.dp))
            .background(MaterialTheme.colorScheme.primaryContainer),
        contentAlignment = Alignment.Center
    ) {
        Text(
            text = value.toString(),
            style = MaterialTheme.typography.displayMedium,
            color = MaterialTheme.colorScheme.onPrimaryContainer
        )
    }
}
```

Line	What it does
List(dice.size) { Random.nextInt(1, 7) }	A new list with a random number from 1 to 6 for each die (7 is excluded)
dice.distinct().size == 1	All dice show the same number: doubles
dice = listOf(1) / listOf(1, 1)	Switches between one and two dice by replacing the list

Tip: To use real dice images, add dice_1.png to dice_6.png in res/drawable and pick one with `when (value) { 1 -> R.drawable.dice_1 ... }` and painterResource (Android only).

Show either app in MainActivity

MainActivity.kt (Android only)

```
setContent {
    Week6PracticeTheme {
        Scaffold { innerPadding ->
            TipCalculatorScreen(modifier = Modifier.padding(innerPadding))
            // or: DiceRollerScreen(modifier = Modifier.padding(innerPadding))
        }
    }
}
```

Project challenges

- Tip Calculator: add quick-pick FilterChips (10%, 15%, 20%) that set the slider value.
- Tip Calculator: show an error when the amount is not a valid number (but not when it's empty).
- Dice Roller: animate the change with **Crossfade(targetState = value)** inside DieFace.
- Dice Roller: keep a history of the last 5 totals in a list in state and show it under the button.

Week 6 checklist

I can...	Done?
Explain state and recomposition	[ ]
Use remember, rememberSaveable and mutableStateOf with by	[ ]
Explain the Activity lifecycle and what rotation does	[ ]
Keep lists in state correctly	[ ]
Build text fields with keyboard options, validation and errors	[ ]
Hoist state to make stateless, reusable composables	[ ]
Use checkbox, switch, radio, slider, dropdown and dialog	[ ]
Add simple animations	[ ]
Build the Tip Calculator and the Dice Roller	[ ]

Quick Quiz

Q1. Why is calculateTip a plain function instead of being inside the composable?

Q2. Why is total not stored in state?

Q3. What does Random.nextInt(1, 7) return?

Q4. Why use rememberSaveable for the bill amount?

Q5. Which composables in the Tip Calculator are stateless?

Answer Key

Try every exercise yourself **before** you look here. Your solution may be different and still correct, as long as the output is right!

Day 1: State & Recomposition

Exercise 1

```
@Composable
fun StepCounter() {
    var steps by remember { mutableStateOf(0) }
    Column(modifier = Modifier.padding(16.dp)) {
        Text("Steps: $steps", style = MaterialTheme.typography.headlineSmall)
        Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
            Button(onClick = { steps += 1 }) { Text("+1") }
            Button(onClick = { steps += 10 }) { Text("+10") }
            OutlinedButton(onClick = { steps = 0 }) { Text("Reset") }
        }
    }
}
```

Exercise 2

```
@Composable
fun LightSwitch() {
    var on by remember { mutableStateOf(false) }
    Column(modifier = Modifier.padding(16.dp)) {
        Box(Modifier.size(80.dp).background(
            if (on) MaterialTheme.colorScheme.tertiaryContainer else MaterialTheme.colorScheme.surfaceVariant))
        Button(onClick = { on = !on }) { Text(if (on) "Turn off" else "Turn on") }
    }
}
```

Exercise 3

```
@Composable
fun WaterTracker() {
    var ml by remember { mutableStateOf(0) }
    Column(modifier = Modifier.padding(16.dp)) {
        Text("$ml ml")
        if (ml >= 2000) Text("Goal reached!", color = MaterialTheme.colorScheme.primary)
        Button(onClick = { ml += 250 }) { Text("Add a glass") }
    }
}
```

Exercise 4

```
// BrokenCounter1: count is a normal variable. Compose doesn't observe it, so changing it
// never triggers recomposition. BrokenCounter2: the state isn't remembered, so every
// recomposition creates a new state starting at 0.
@Composable
fun FixedCounter() {
    var count by remember { mutableStateOf(0) }
    Button(onClick = { count++ }) { Text("Count: $count") }
}
```

Quiz answers

Q1. Compose re-running a composable function because state it reads has changed.

Q2. Keeps a value across recompositions.

Q3. Compose doesn't observe it, so changing it doesn't trigger recomposition.

Q4. No. Calculate it from quantity and price (a derived value) so it can't get out of sync.

Q5. In event handlers such as onClick, never directly during composition.

Day 2: rememberSaveable & the Activity Lifecycle

Exercise 1

```
@Composable
fun SaveableStepCounter() {
    var steps by rememberSaveable { mutableStateOf(0) }
    Button(onClick = { steps++ }) { Text("Steps: $steps") }
}
```

Exercise 2

```
// Opening: onCreate, onStart, onResume. Home: onPause, onStop.
// Returning: onRestart, onStart, onResume.
// Rotating: onPause, onStop, onDestroy, onCreate, onStart, onResume.
@Composable
fun LifecycleNote() { Text("See the comment above") }
```

Exercise 3

```
@Composable
fun TodoQuickList() {
    val tasks = remember { mutableStateListOf<String>() }
    Column(modifier = Modifier.padding(16.dp)) {
        Button(onClick = { tasks.add("Task ${tasks.size + 1}") }) { Text("Add task") }
        tasks.forEach { task ->
            Row(verticalAlignment = Alignment.CenterVertically) {
                Text(task, modifier = Modifier.weight(1f))
                TextButton(onClick = { tasks.remove(task) }) { Text("Remove") }
            }
        }
    }
}
```

Exercise 4

```
@Composable
fun TodoQuickListImmutable() {
    var tasks by rememberSaveable { mutableStateOf(listOf<String>()) }
    var counter by rememberSaveable { mutableStateOf(0) }
    Column(modifier = Modifier.padding(16.dp)) {
        Button(onClick = { counter++; tasks = tasks + "Task $counter" }) { Text("Add task") }
        tasks.forEach { task ->
            Row(verticalAlignment = Alignment.CenterVertically) {
                Text(task, modifier = Modifier.weight(1f))
                TextButton(onClick = { tasks = tasks - task }) { Text("Remove") }
            }
        }
    }
}
```

Quiz answers

Q1. Rotation destroys and recreates the Activity, and remember only survives recomposition.

Q2. rememberSaveable (or a ViewModel).

Q3. onPause, onStop, onDestroy, then onCreate, onStart, onResume.

Q4. A normal list isn't observable; Compose doesn't know it changed.

Q5. mutableStateListOf, or an immutable list in mutableStateOf that you replace with a new list.

Day 3: Text Input

Exercise 1

```
@Composable
fun BmiInput() {
    var weight by rememberSaveable { mutableStateOf("") }
    var height by rememberSaveable { mutableStateOf("") }
    val w = weight.toDoubleOrNull()
    val h = height.toDoubleOrNull()?.div(100)
    Column(modifier = Modifier.padding(16.dp)) {
        OutlinedTextField(weight, { weight = it }, label = { Text("Weight (kg)") },
            keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal))
        OutlinedTextField(height, { height = it }, label = { Text("Height (cm)") },
            keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal))
        if (w != null && h != null && h > 0) {
            Text("BMI: ${"%.1f".format(w / (h * h))}")
        }
    }
}
```

Exercise 2

```
@Composable
fun FeedbackField() {
    var text by rememberSaveable { mutableStateOf("") }
    OutlinedTextField(
        value = text,
        onValueChange = { if (it.length <= 200) text = it },
        label = { Text("Feedback") },
        minLines = 3,
        maxLines = 5,
        supportingText = { Text("${text.length} / 200") },
        modifier = Modifier.fillMaxWidth().padding(16.dp)
    )
}
```

Exercise 3

```
@Composable
fun PasswordPair() {
    var password by rememberSaveable { mutableStateOf("") }
    var confirm by rememberSaveable { mutableStateOf("") }
    val mismatch = confirm.isNotEmpty() && confirm != password
    Column(modifier = Modifier.padding(16.dp)) {
        OutlinedTextField(password, { password = it }, label = { Text("Password") },
            visualTransformation = PasswordVisualTransformation())
        OutlinedTextField(confirm, { confirm = it }, label = { Text("Confirm password") },
            visualTransformation = PasswordVisualTransformation(),
            isError = mismatch,
            supportingText = { if (mismatch) Text("Passwords don't match") })
    }
}
```

Exercise 4

```
@Composable
fun PhoneField() {
    var phone by rememberSaveable { mutableStateOf("") }
    val error = phone.length != 10
    OutlinedTextField(
        value = phone,
        onValueChange = { if (it.all(Char::isDigit) && it.length <= 10) phone = it },
        label = { Text("Phone number") },
        isError = error,
        supportingText = { if (error) Text("Enter 10 digits") },
        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Phone),
        singleLine = true
    )
}
```

Quiz answers

Q1. The field doesn't change when the user types.

Q2. keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number)

Q3. keyboardActions = KeyboardActions(onDone = { focusManager.clearFocus() })

Q4. The text can be empty or invalid; toIntOrNull returns null instead of crashing.

Q5. isError = true, and supportingText for the message.

Day 4: State Hoisting

Exercise 1

```
@Composable
fun LikeButton(liked: Boolean, likes: Int, onToggle: () -> Unit) {
    Row(verticalAlignment = Alignment.CenterVertically) {
        IconButton(onClick = onToggle) {
            Icon(if (liked) Icons.Filled.Favorite else Icons.Outlined.FavoriteBorder,
                contentDescription = if (liked) "Unlike" else "Like")
        }
        Text("$likes likes")
    }
}

@Composable
fun PostScreen() {
    var liked by rememberSaveable { mutableStateOf(false) }
    var likes by rememberSaveable { mutableStateOf(41) }
    LikeButton(liked, likes, onToggle = {
        liked = !liked
        likes += if (liked) 1 else -1
    })
}
```

Exercise 2

```
@Composable
fun RatingBar(rating: Int, onRatingChange: (Int) -> Unit) {
    Row {
        for (star in 1..5) {
            IconButton(onClick = { onRatingChange(star) }) {
                Icon(if (star <= rating) Icons.Filled.Star else Icons.Outlined.Star,
                    contentDescription = "$star stars")
            }
        }
    }
}

@Composable
fun RatingScreen() {
    var rating by rememberSaveable { mutableStateOf(0) }
    Column(modifier = Modifier.padding(16.dp)) {
        RatingBar(rating, onRatingChange = { rating = it })
        Text("You rated $rating/5")
    }
}
```

Exercise 3

```
@Composable
fun RupeeInput(value: String, onValueChange: (String) -> Unit) {
    OutlinedTextField(value, onValueChange, label = { Text("Rupees") },
        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal))
}

@Composable
fun DollarResult(rupees: Double?) {
    Text(if (rupees == null) "Enter an amount" else "= $${"%.2f".format(rupees / 84)}")
}

@Composable
fun CurrencyConverter() {
    var input by rememberSaveable { mutableStateOf("") }
    Column(modifier = Modifier.padding(16.dp)) {
        RupeeInput(input, { input = it })
        DollarResult(input.toDoubleOrNull())
    }
}
```

Exercise 4

```
@Composable
fun QuantitySelector(quantity: Int, onQuantityChange: (Int) -> Unit, min: Int = 0) {
    Row(verticalAlignment = Alignment.CenterVertically) {
        IconButton(onClick = { onQuantityChange(quantity - 1) }, enabled = quantity > min) {
            Icon(Icons.Default.KeyboardArrowDown, contentDescription = "Decrease")
        }
        Text("$quantity")
        IconButton(onClick = { onQuantityChange(quantity + 1) }) {
            Icon(Icons.Default.KeyboardArrowUp, contentDescription = "Increase")
        }
    }
}

@Composable
fun TicketScreen() {
    var adults by rememberSaveable { mutableStateOf(1) }
    var children by rememberSaveable { mutableStateOf(0) }
    Column(modifier = Modifier.padding(16.dp)) {
        Row(verticalAlignment = Alignment.CenterVertically) {
            Text("Adults", Modifier.weight(1f)); QuantitySelector(adults, { adults = it })
        }
        Row(verticalAlignment = Alignment.CenterVertically) {
            Text("Children", Modifier.weight(1f)); QuantitySelector(children, { children = it })
        }
        Text("Total: Rs ${adults * 200 + children * 100}")
    }
}
```

Quiz answers

Q1. Moving state up to a parent and passing the value and an event callback down to the child.

Q2. In the lowest common parent of all composables that read or change it.

Q3. Reusable, easy to preview, easy to test, controllable from outside.

Q4. Each piece of state is stored in exactly one place.

Q5. The value (e.g. checked) and an event (e.g. onCheckedChange).

Day 5: Interactive Components

Exercise 1

```
@Composable
fun PizzaOrder() {
    var size by rememberSaveable { mutableStateOf("Medium") }
    var cheese by rememberSaveable { mutableStateOf(false) }
    var spicy by rememberSaveable { mutableStateOf(1f) }
    Column(modifier = Modifier.padding(16.dp)) {
        listOf("Small", "Medium", "Large").forEach { option ->
            Row(verticalAlignment = Alignment.CenterVertically) {
                RadioButton(selected = size == option, onClick = { size = option }); Text(option)
            }
        }
        Row(verticalAlignment = Alignment.CenterVertically) {
            Checkbox(cheese, { cheese = it }); Text("Extra cheese")
        }
        Text("Spicy level: ${spicy.roundToInt()}")
        Slider(value = spicy, onValueChange = { spicy = it }, valueRange = 0f..3f, steps = 2)
        Text("$size pizza${if (cheese) " with extra cheese" else ""}, spicy ${spicy.roundToInt()}")
    }
}
```

Exercise 2

```
@Composable
fun DeliveryScreen() {
    var city by rememberSaveable { mutableStateOf("Kochi") }
    Column(modifier = Modifier.padding(16.dp)) {
        CityDropdown(selected = city, onSelect = { city = it })
        Text("Delivering to: $city")
    }
}

@Composable
fun CityDropdown(selected: String, onSelect: (String) -> Unit) {
    // the CityDropdown from section 3
    Text(selected, modifier = Modifier.clickable { onSelect("Thrissur") })
}
```

Exercise 3

```
@Composable
fun DeleteAccountButton(onDelete: () -> Unit) {
    var show by remember { mutableStateOf(false) }
    var typed by remember { mutableStateOf("") }
    Button(onClick = { show = true }) { Text("Delete account") }
    if (show) {
        AlertDialog(
            onDismissRequest = { show = false },
            title = { Text("Delete account?") },
            text = {
                OutlinedTextField(typed, { typed = it }, label = { Text("Type DELETE") })
            },
            confirmButton = {
                TextButton(enabled = typed == "DELETE", onClick = { show = false; onDelete() }) {
                    Text("Delete")
                }
            },
            dismissButton = { TextButton(onClick = { show = false }) { Text("Cancel") } }
        )
    }
}
```

Exercise 4

```
@Composable
fun FaqList() {
    Column {
        ExpandableFaq("Is the app free?", "Yes, all features are free.")
        ExpandableFaq("Does it work offline?", "Yes, your data is stored on the phone.")
        ExpandableFaq("How do I reset my password?", "Tap 'Forgot password?' on the login screen.")
    }
}

@Composable
fun ExpandableFaq(question: String, answer: String) {
    var expanded by rememberSaveable { mutableStateOf(false) }
    Column(modifier = Modifier.clickable { expanded = !expanded }.padding(16.dp)) {
        Text(question)
        AnimatedVisibility(visible = expanded) { Text(answer) }
    }
}
```

Quiz answers

Q1. The small checkbox is hard to tap; the whole row is a bigger target.

Q2. 9 stops between the ends, so the value moves in jumps of 10.

Q3. Put it inside if (showDialog) and change the Boolean state.

Q4. expanded is local UI state; the selected city is hoisted.

Q5. AnimatedVisibility

Day 6: Weekly Projects: Tip Calculator & Dice Roller

Quiz answers

Q1. Logic is easier to test and reuse when it's separate from UI.

Q2. It's derived from amount and tip, so it's calculated on every recomposition.

Q3. A random Int from 1 to 6 (the upper bound is excluded).

Q4. So the user's input survives rotation.

Q5. QuantitySelector and ResultCard.

> **Next:** When you finish this week, tell your mentor and ask for the **Week 7** material. Great work, Anjana!